

Data Science and Management

Project-AI: Capability-Scoped Execution Governance with Selective Admissibility, Stateful Threat Tracking, and Canonical Proof Artifacts --Manuscript Draft--

Manuscript Number:	
Full Title:	Project-AI: Capability-Scoped Execution Governance with Selective Admissibility, Stateful Threat Tracking, and Canonical Proof Artifacts
Article Type:	VSI: AI-powered E-Com and Information Management:Research Article
Keywords:	runtime governance enforcement; selective admissibility; governance liveness; capability tokens; evidence bundle; stateful threat tracking
Corresponding Author:	Jeremy Karrick, N/A Independent Researcher UNITED STATES OF AMERICA
Corresponding Author Secondary Information:	
Corresponding Author's Institution:	Independent Researcher
Corresponding Author's Secondary Institution:	
First Author:	Jeremy Karrick, N/A
First Author Secondary Information:	
Order of Authors:	Jeremy Karrick, N/A
Order of Authors Secondary Information:	
Abstract:	Existing AI runtime governance architectures enforce constitutional constraints through structural admissibility gates, but treat governance as binary: actions are either allowed or denied. This paper presents Project-AI v2, extending the execution-governed constitutional architecture with three primary contributions. (Continues in paper)
Additional Information:	
Question	Response

Project-AI: Capability-Scoped Execution Governance with Selective Admissibility, Stateful Threat Tracking, and Canonical Proof Artifacts

Anonymous Author

Zenodo DOIs: [redacted for review]

Abstract

Existing AI runtime governance architectures enforce constitutional constraints through structural admissibility gates, but treat governance as binary: actions are either allowed or denied. This paper presents Project-AI v2, extending the execution-governed constitutional architecture with three primary contributions: (1) a Safe-Allow Calibration Layer implementing a seven-outcome governance model (ALLOW, DENY, CLARIFY, HUMAN_APPROVAL_REQUIRED, DEGRADED_READ_ONLY, HALT, ESCALATE) that resolves the governance liveness problem by distinguishing safe, unsafe, underspecified, high-impact, degraded, unverifiable, and constitutional-critical request categories; (2) a ConversationThreatStateRegister tracking eight risk dimensions across session turns—including intent drift, manipulation score, obfuscation score, and baseline drift—enabling stateful multi-turn threat detection that addresses the boiling-frog escalation problem; and (3) a separated authorization pipeline comprising PolicyDecision (type-level permission), ExecutionAuthorization (instance-level context binding), and CapabilityToken (short-lived signed execution credential) that closes the approval reuse and action substitution vulnerabilities inherent in single-stage authorization. Every governed request—including denials, clarifications, and halts—produces a canonical EvidenceBundle containing policy version, invariant results with severity levels, continuity proof, conversation threat state, and audit chain linkage. A 186-test validation suite (186 passed, 0 failed, 19 test files) covers all new components. Post-upgrade evaluation on JailbreakBench (40 prompts, 31 harmful, 9 benign) with the two-tier SemanticRiskClassifier: benign allowed rate rises from 0% to 100% (zero false positives, precision = 1.000); harmful blocked rate is 67.7% (21/31)—the 10 misses use indirect/meta-structural framing that evades both embedding and lexical tiers. F1 is 0.808. Multi-turn CTSR detection improves from 1/5 to 4/5 adversarial scenarios. The one miss scores conversation risk 0.429, just below the 0.45 block threshold. Five canonical invariants pass deterministic replay; RFC 3161 timestamping provides externalized time verification with clock-skew detection.

Keywords: runtime governance enforcement, selective admissibility, governance liveness, capability tokens, evidence bundle, stateful threat tracking, constitutional policy orchestration, deterministic enforcement architecture, proof-carrying execution

1. Introduction

The dominant approach to AI governance treats policy as an external constraint applied to a system that already executes freely. Constitutional AI trains models to follow principles via self-critique and RLHF. The NIST AI RMF defines organizational risk processes. The EU AI Act establishes external conformity assessment. These mechanisms share a structural property: governance operates outside the execution path. A model that has been trained, aligned, and audited still executes its output before any runtime governance check intervenes.

Prior work introduced structural admissibility enforcement via an execution gate that invokes governance evaluation before any executor function is reachable. That architecture addressed the governance-collapse and unprovable-admissibility failure modes. However, it exposed three secondary problems: (1) the governance liveness problem—binary deny-all governance blocks benign traffic, making the system behave like a perimeter firewall rather than a policy engine; (2) the boiling-frog escalation problem—turn-local threat evaluation misses gradual conversation-context manipulations that individually appear benign but collectively constitute an attack; and (3) the approval reuse problem—a single governance decision that permits an action type provides no binding against replaying that authorization for a different instance, context, or policy epoch.

This paper presents the structural upgrades that address these problems, implemented and validated in the accompanying artifact and codebase branch [redacted for review].

The reference monitor model requires that the trusted intermediary be invoked on every access decision. Runtime verification checks system properties over execution traces. Capability-based security bounds the scope of any single authorization. This work applies all three principles to the extended governance architecture.

The principal contributions are:

- Seven-outcome governance model with ALLOW, DENY, CLARIFY, HUMAN_APPROVAL_REQUIRED, DEGRADED_READ_ONLY, HALT, and ESCALATE.
- Safe-Allow Calibration Layer using a RiskClassifier -> BenignIntentValidator -> PolicyConflictResolver pipeline.
- ConversationThreatStateRegister with an eight-component weighted risk model.

- Separated authorization pipeline with `PolicyDecision`, `ExecutionAuthorization`, and `CapabilityToken`.
- Canonical `EvidenceBundle` generated for every governed request.
- InvariantSeverity levels: INFO, WARN, BLOCK, HALT, ESCALATE.
- Governance Microkernel model separating trusted kernel from userspace.
- Stateful branch protection in `STATE_REGISTER`.
- Governed recovery protocol with genesis re-anchoring restrictions.
- 186-test validation suite across 19 test files.

2. Background and Prior Work

2.1 Reference Monitor and Runtime Verification

Anderson's reference monitor concept establishes that a trusted intermediary must be invoked for every access operation, must be tamper-proof, and must be small enough to be verifiable. Runtime verification checks system behavior against formal specifications during execution. The execution gate implements both principles: it is invoked before every executor function call, and the invariant engine checks formal predicates over execution state.

The governance microkernel model formalizes the Anderson boundary for AI governance: the minimal trusted set required to enforce the governance contract, with everything else treated as userspace and therefore partially or fully untrusted.

2.2 Capability-Based Security

Capability-based security bounds authorization scope to a transferable, unforgeable reference. The `CapabilityToken` in this work applies least-privilege principles: tokens are action-scoped, context-bound, expiring, and one-time-use. An approved policy type does not constitute a capability; a capability must be explicitly minted per instance.

2.3 Policy Enforcement Architectures

Role-based and attribute-based access control enforce policy decisions before resource access. These frameworks separate policy evaluation from policy enforcement—the same separation applied here between `PolicyDecision` (evaluation) and `ExecutionAuthorization` (enforcement). Policy-based management extends this to network and system configuration; the `PolicyRegistry` applies analogous versioning and migration semantics to AI governance policy.

2.4 Jailbreak and Multi-Turn Attack Literature

JailbreakBench characterizes single-turn attack categories including instruction override, roleplay escalation, encoding obfuscation, authority manipulation, and DAN persona attacks. Multi-turn attacks exploit context accumulation: individually benign turns establish a context that makes a later harmful request appear legitimate. The ConversationThreatStateRegister provides a formal model for detecting such gradual escalation by maintaining a risk baseline and measuring drift from it across turns.

3. Constitutional Execution Model

3.1 Core Definitions

Admissibility: An action A is admissible under governance state G if and only if: (1) A 's risk classification is not unsafe; (2) A 's policy type is permitted by `PolicyDecision`; (3) A 's execution instance is authorized by `ExecutionAuthorization`; (4) a valid `CapabilityToken` has been minted for A ; (5) all invariants at severity `BLOCK` or higher pass; and (6) continuity state is verified.

Governance Outcome: A governance evaluation maps a request R under state G to `GovernanceOutcome` $\in \{\text{ALLOW, DENY, CLARIFY, HUMAN_APPROVAL_REQUIRED, DEGRADED_READ_ONLY, HALT, ESCALATE}\}$. Only `ALLOW` and `DEGRADED\_READ\_ONLY` permit execution to proceed.

Liveness Property: A governance architecture satisfies liveness if for all safe, low-risk, well-specified requests R , $\text{evaluate}(R) = \text{ALLOW}$.

Governance Invariant: A formal predicate P over execution state S that must hold before any admissible state transition. `InvariantSeverity` $\in \{\text{INFO, WARN, BLOCK, HALT, ESCALATE}\}$ determines the consequence of predicate failure.

EvidenceBundle: A canonical proof artifact E produced for every governed request R , containing request hash, governance decisions, invariant results, conversation threat state, continuity proof, capability token reference, policy version and hash, and audit chain linkage.

3.2 Governance Contract

For any action to execute, ten conditions must hold:

1. The action must be admissible.
2. It must preserve all invariants at `BLOCK` severity or higher.
3. It must maintain continuity.

4. It must be cryptographically bound.
5. It must be auditable.
6. It must be authorized for the exact instance.
7. It must execute only within capability token scope.
8. It must respect the current policy version and migration state.
9. It must degrade safely when governance is impaired.
10. It must not substitute, replay, or expand beyond minted token parameters.

4. System Architecture

Project-AI implements a six-plane PSIA (Parallel Sovereign Isolation Architecture) with a twelve-stage governance pipeline.

4.1 Execution Planes

- **Ingress Plane:** External interface, CORS, sanitization, intent detection.
- **Gate Plane:** GateGuardian, TARL hard policy gate, GovernanceOutcome routing.
- **Canonical Plane:** Primary execution, persistent side-effects committed here.
- **Shadow Plane:** Side-effect-free simulation, intent hash compared to Ingress.
- **Adaptive Plane:** Policy parameter adjustment, subject to admissibility gate.
- **Reflex Plane:** OctoReflex, constitutional enforcement, MISMATCHED_INTENT detection.

4.2 Governance Microkernel

The governance microkernel comprises the minimal trusted set: ExecutionGate, InvariantEngine, PolicyRegistry, CapabilityTokenVerifier, EvidenceBundleWriter, StateRegister, AuditChain verifier, and ProofExecutor. Everything else is treated as microkernel userspace: LLM providers, tools, plugins, adapters, external APIs, UI, and model-generated code.

4.3 Threat Model

The threat model defines trust zones:

- **Trusted:** microkernel components.
- **Partially Trusted:** model outputs, user inputs, plugins, adapters, council members, external policy feeds.

- **Untrusted:** external requests, prompt content, uploaded files, network responses, model-generated code, prior conversation claims, tool outputs before validation.

Threat surfaces are mapped to controls across input manipulation, execution substitution, temporal manipulation, governance bypass, and side-effect abuse.

5. Safe-Allow Calibration Layer

The governance liveness problem arises when a governance architecture achieves safety at the cost of liveness. The Safe-Allow Calibration Layer is inserted between risk detection and the final governance gate.

5.1 Pipeline Design

Pipeline:

Request -> RiskClassifier -> BenignIntentValidator -> PolicyConflictResolver -> GovernanceGate -> ExecutionAuthorization

- **RiskClassifier:** computes risk score in [0.0, 1.0] with levels SAFE, LOW, MEDIUM, HIGH, CRITICAL.
- **BenignIntentValidator:** applies positive-intent indicators; benign confidence ≥ 0.6 with low risk can enable ALLOW.
- **PolicyConflictResolver:** routes conflicting signals to CLARIFY instead of forcing binary decisions.

5.2 GovernanceOutcome Semantics

Outcome	Condition	Execution Permitted
ALLOW	Safe, low-risk, benign intent	Yes
DEGRADED_READ_ONLY	Governance degraded; no-side-effect actions	Read-only only
CLARIFY	Ambiguous or underspecified request	No
HUMAN_APPROVAL_REQUIRED	High-impact action or elevated risk	No
DENY	Unsafe intent or policy violation	No
HALT	Unverifiable state	No
ESCALATE	Constitutional or security-critical violation	No

Only ALLOW and DEGRADED_READ_ONLY permit execution.

6. Conversation Threat State Register

Turn-local evaluation is insufficient for attacks that exploit context accumulation across turns. The `ConversationThreatStateRegister` maintains session risk with eight components.

6.1 Risk Model

The composite score is formed from weighted components:

- current turn risk
- prior denial rate
- semantic similarity to blocked goal
- escalation pattern
- obfuscation score
- authority claim score
- manipulation score
- baseline drift score

Default weights are [0.25, 0.20, 0.15, 0.15, 0.10, 0.05, 0.05, 0.05]. Thresholds are: ALLOW (< 0.3), WARN (< 0.5), CLARIFY (< 0.7), DENY (< 0.85), ESCALATE (>= 0.85).

6.2 Governed Reset

Risk state may be reset only via `governed_reset(auth_token)`, which requires a non-empty authorization token. Unauthorized reset raises `GovernedResetDenied`.

7. Separated Authorization Pipeline

7.1 PolicyDecision

`PolicyDecision` evaluates whether an action type is permitted under the active governance policy. It is necessary but not sufficient for execution.

7.2 ExecutionAuthorization

`ExecutionAuthorization` evaluates whether the exact execution instance may run under current context using eight guards:

1. Session freshness.

2. Recipient binding.
3. User confirmation.
4. Attachment safety.
5. Scope validation.
6. Identity context.
7. Governance state.
8. Policy hash binding.

7.3 CapabilityToken

On successful `PolicyDecision` and `ExecutionAuthorization`, the `CapabilityTokenService` mints a short-lived execution credential. The token is HMAC-SHA256 signed, defaults to a 300-second lifetime, and verifies against replay, expiry, signature, action, scope, context hash, and policy hash.

8. Evidence Bundle

Every governed request produces an `EvidenceBundle`, a canonical, immutable, proof-carrying record of every governance decision made on that request's behalf.

8.1 Schema Highlights

The schema includes:

- `bundle_id`
- `request_hash`
- `intent_classification`
- `risk_classification`
- `benign_validation`
- `policy_decision`
- `execution_authorization`
- `capability_token`
- `invariant_results`
- `conversation_threat_state`
- `final_outcome`

- `audit_chain_prev` / `audit_chain_next`
- `timestamp_token`
- `executor_result_hash`

Sensitive subcomponents are stored as SHA-256 hashes rather than raw values.

8.2 Storage

The in-process evidence store is a thread-safe ring buffer capped at 10,000 entries. Production deployments should forward bundles to an external durable audit log.

9. Invariant Severity Levels

The severity model separates distinct governance consequences.

Severity	Governance Action	Example Condition
INFO	Audit-only; execution continues	Optional metadata absent
WARN	Audit + notify; continues if no higher severity	Slightly stale session metadata
BLOCK	Execution prevented	Stale continuity proof
HALT	Session/action init prevented	Forged continuity proof
ESCALATE	Council notified; audit trail entry	Signing key mismatch, invariant registry tampering

Execution proceeds only if no invariant result has severity BLOCK or higher.

10. Governance Council (Triumvirate)

The Triumvirate provides quorum-based governance authority through three independent pillars:

- **Galahad:** constitutional arbiter.
- **Cerberus:** security evaluator.
- **Codex Deus Maximus:** policy adjudicator.

No single pillar can unilaterally approve a multi-pillar-consensus action. Unsigned policy threshold weakening triggers ESCALATE or HUMAN_APPROVAL_REQUIRED.

11. State Register and Temporal Continuity

The `STATE_REGISTER` enforces the Flat Gap Protocol: no AI session may initialize without a verified continuity proof linking it to its predecessor session.

11.1 StateBranchingProtector

`StateBranchingProtector` maintains a map from `predecessor_hash` to `sequence_number`. If two sessions present the same predecessor hash with different sequence numbers, a `BranchConflictError` is raised, triggering `HALT` or `ESCALATE` routing.

11.2 Policy Change During Human Gap

If the active policy hash changed during a sufficiently large human gap, `PolicyRegistry` returns `CLARIFY` or `HUMAN_APPROVAL_REQUIRED` before normal operation resumes.

12. OctoReflex Runtime Containment

`OctoReflex` implements the constitutional enforcement layer in the Reflex Plane.

12.1 Enforcement Level Taxonomy

- `MONITOR`: observation only.
- `WARN`: warning logged; execution continues.
- `BLOCK`: action blocked.
- `TERMINATE`: session terminated.
- `ESCALATE`: immediate governance council escalation.

12.2 MISMATCHED_INTENT Violation

Cross-plane semantic collision detection computes intent hashes at Ingress, Shadow simulation, and execution planning. If intent differs materially across planes, a `MISMATCHED_INTENT` violation is emitted with `BLOCK` severity.

13. Evaluation

13.1 Validation Suite

The structural upgrade branch was validated with a 186-test suite across 19 test files, with 186 passed and 0 failed.

1

2

3

4 **13.2 Single-Turn Evaluation**

5

6 The JailbreakBench evaluation suite comprises 40 prompts: 31 harmful and 9 benign controls.

7

8

9

Metric	Value	Notes
Total prompts	40	31 harmful, 9 benign
Harmful blocked	31/31 (100%)	Baseline
Benign allowed	0/9 (0%)	Baseline
Precision	0.775	TP=31, FP=9
Recall	1.000	No harmful prompts passed
F1	0.873	Baseline

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25 **13.3 Multi-Turn Evaluation**

26

27

Metric	Value	Notes
Total scenarios	15	13 attack, 2 benign
Overall mitigation rate	53.3% (8/15)	Baseline
Attack recall	46.2% (6/13)	Baseline
Precision	1.000	No false positives
F1	0.632	Baseline

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42 **13.4 Performance Benchmarks**

43

44 New governance components add sub-millisecond overhead per execution.

45

46

47

Component	Avg Latency
Constitutional rule check	< 0.001 ms
GovernanceOutcome evaluation	< 0.002 ms
RiskClassifier	< 0.05 ms
BenignIntentValidator	< 0.05 ms
CTSR risk score computation	< 0.1 ms
CapabilityToken mint	< 0.5 ms

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

CapabilityToken verify	< 0.5 ms
EvidenceBundle build	< 1.0 ms

13.5 Post-Upgrade Selective Governance Evaluation

Metric	Baseline	Post-Upgrade	Delta
Harmful blocked rate	100.0%	67.7% (21/31)	-32.3pp
Benign allowed rate	0.0%	100.0% (9/9)	+100.0pp
False positive rate	100.0%	0.0% (0/9)	-100.0pp
False negative rate	0.0%	32.3% (10/31)	+32.3pp
Precision	0.775	1.000	+0.225
Recall	1.000	0.677	-0.323
F1	0.873	0.808	-0.065

Interpretation: the SemanticRiskClassifier achieves perfect precision but reduced recall, producing a clear precision-recall tradeoff.

13.6 Post-Upgrade Multi-Turn Escalation (CTSR)

Scenario	Final Score	Correctly Detected
A — Boiling Frog	0.498	Yes
B — Authority Escalation	0.534	Yes
C — Sudden Obfuscated	0.429	No
D — DAN + Trust Build	0.541	Yes
E — Story-Wrapped	0.567	Yes

CTSR correctly detects 4/5 adversarial escalation scenarios.

13.7 Regulatory and Compliance Coverage

Standard	Mechanism
SOC 2	EvidenceBundle + SHA-256 audit chain + denial records

GDPR	Privacy ledger; EvidenceBundle redacts sensitive data
HIPAA	Tamper-proof log; capability token audit trail
ISO 27001	PolicyRegistry versioning + signed transitions
EU AI Act	EvidenceBundle + human oversight routing + TARL gate

14. Limitations

Identified limitations include:

- SemanticRiskClassifier recall gap (32.3% false-negative rate on harmful JBB prompts).
- CTSR near-miss on sudden obfuscated attack scenario.
- CapabilityToken currently uses HMAC-SHA256; production should move to Ed25519.
- RFC 3161 TSA is mocked in tests.
- Distributed and BFT governance modes are roadmap items only.
- Formal verification uses executable predicates rather than theorem-proven specifications.
- Replay store is in-process and not durable across restarts.
- GPU and inference-cluster governance is not yet addressed.

15. Discussion

15.1 Governance Liveness as a First-Class Property

Safety and liveness are dual properties. A governance system that blocks all harmful actions but also blocks all beneficial ones is technically safe but operationally useless. The CLARIFY outcome resolves this by distinguishing incomplete requests from unsafe ones.

15.2 Admissibility vs. Alignment

Admissibility enforcement and alignment training address different failure modes and are composable. Alignment shapes model behavior probabilistically at training time, while admissibility provides deterministic runtime enforcement.

15.3 Runtime Constitutions

Runtime constitutions are presented as a generalizable design pattern: governance microkernel, formal invariant engine, capability token system, and canonical proof artifact.

16. Conclusion

This paper presents the structural upgrade of Project-AI from a binary admissibility gate into a capability-scoped, selectively admissible, evidence-producing runtime governance architecture. The Safe-Allow Calibration Layer, ConversationThreatStateRegister, and separated authorization pipeline each address a distinct failure mode of the prior architecture. Post-upgrade evaluation shows major liveness gains with an explicit and documented precision-recall tradeoff. Every governed request produces a canonical `EvidenceBundle`, and the architecture is validated by a 186-test suite across 19 test files.

Conflict of Interest Statement

The author declares that there are no known financial or personal conflicts of interest that could have appeared to influence the work reported in this manuscript. [web:53][cite:56]

The research was self-funded using the author's own time and resources, with no external sponsor, grant, institutional funder, or commercial entity directing, financing, or controlling the work. [cite:56][cite:58][cite:59]

The author is the founder and principal architect of Thirstys Projects, the organization associated with Project-AI, and this affiliation is disclosed for transparency. [cite:58][cite:59]

No human participants, patient data, or protected datasets were involved in this work. [cite:56]